torch.dynamic-shape#

https://pytorch.org/docs/stable/generated/exportdb/torch.dynamic-shape.html

Description:

Tags: torch.cond, torch.dynamic-shape Support Level: SUPPORTED

Original source code:

Code Examples

```
# mypy: allow-untyped-defs
import torch

from functorch.experimental.control_flow import cond

class MySubModule(torch.nn.Module):
    def foo(self, x):
        return x.cos()

    def forward(self, x):
        return self.foo(x)

class CondBranchClassMethod(torch.nn.Module):
    """
    The branch functions (`true_fn` and `false_fn`) passed to
cond() must follow these rules:
      - both branches must take the same args, which must also
```

```
match the branch args passed to cond.
      - both branches must return a single tensor
      - returned tensor must have the same tensor metadata, e.g.
shape and dtype
      - branch function can be free function, nested function,
lambda, class methods
      - branch function can not have closure variables
      - no inplace mutations on inputs or global variables


    This example demonstrates using class method in cond().

    NOTE: If the `pred` is test on a dim with batch size < 2, it
will be specialized.
    """

    def __init__(self) -> None:
        super().__init__()
        self.subm = MySubModule()

    def bar(self, x):
        return x.sin()

    def forward(self, x):
        return cond(x.shape[0] <= 2, self.subm.forward,
self.bar, [x])

example_args = (torch.randn(3),)
tags = {
    "torch.cond",
    "torch.dynamic-shape",
}
model = CondBranchClassMethod()


torch.export.export(model, example_args)
```

```
ExportedProgram:
    class GraphModule(torch.nn.Module):
        def forward(self, x: "f32[3]"):
                sin: "f32[3]" = torch.ops.aten.sin.default(x);
x = None
            return (sin,)

Graph signature:
    # inputs
    x: USER_INPUT

    # outputs
    sin: USER_OUTPUT

Range constraints: {}
```

```
# mypy: allow-untyped-defs
import torch

from functorch.experimental.control_flow import cond

class CondBranchNestedFunction(torch.nn.Module):
    """
    The branch functions (`true_fn` and `false_fn`) passed to
cond() must follow these rules:
      - both branches must take the same args, which must also
match the branch args passed to cond.
      - both branches must return a single tensor
      - returned tensor must have the same tensor metadata, e.g.
shape and dtype
      - branch function can be free function, nested function,
lambda, class methods
      - branch function can not have closure variables
      - no inplace mutations on inputs or global variables
```

```
    This example demonstrates using nested function in cond().

    NOTE: If the `pred` is test on a dim with batch size < 2, it
will be specialized.
    """

    def forward(self, x):
        def true_fn(x):
            def inner_true_fn(y):
                return x + y

            return inner_true_fn(x)

        def false_fn(x):
            def inner_false_fn(y):
                return x - y

            return inner_false_fn(x)

        return cond(x.shape[0] < 10, true_fn, false_fn, [x])

example_args = (torch.randn(3),)
tags = {
    "torch.cond",
    "torch.dynamic-shape",
}
model = CondBranchNestedFunction()


torch.export.export(model, example_args)
```

```
ExportedProgram:
    class GraphModule(torch.nn.Module):
        def forward(self, x: "f32[3]"):
                add: "f32[3]" = torch.ops.aten.add.Tensor(x,
x);  x = None
            return (add,)

Graph signature:
    # inputs
    x: USER_INPUT

    # outputs
    add: USER_OUTPUT

Range constraints: {}
```

```
# mypy: allow-untyped-defs
import torch

from functorch.experimental.control_flow import cond

class CondBranchNonlocalVariables(torch.nn.Module):
    """
    The branch functions (`true_fn` and `false_fn`) passed to
cond() must follow these rules:
    - both branches must take the same args, which must also
match the branch args passed to cond.
    - both branches must return a single tensor
    - returned tensor must have the same tensor metadata, e.g.
shape and dtype
    - branch function can be free function, nested function,
lambda, class methods
    - branch function can not have closure variables
    - no inplace mutations on inputs or global variables
```

```
    This example demonstrates how to rewrite code to avoid
capturing closure variables in branch functions.

    The code below will not work because capturing closure
variables is not supported.
    ```
    my_tensor_var = x + 100
    my_primitive_var = 3.14

    def true_fn(y):
        nonlocal my_tensor_var, my_primitive_var
        return y + my_tensor_var + my_primitive_var

    def false_fn(y):
        nonlocal my_tensor_var, my_primitive_var
        return y - my_tensor_var - my_primitive_var

    return cond(x.shape[0] > 5, true_fn, false_fn, [x])
    ```

    NOTE: If the `pred` is test on a dim with batch size < 2, it
will be specialized.
    """

    def forward(self, x):
        my_tensor_var = x + 100
        my_primitive_var = 3.14

        def true_fn(x, y, z):
            return x + y + z

        def false_fn(x, y, z):
            return x - y - z

        return cond(
            x.shape[0] > 5,
            true_fn,
```

```
            false_fn,
            [x, my_tensor_var, torch.tensor(my_primitive_var)],
        )

example_args = (torch.randn(6),)
tags = {
    "torch.cond",
    "torch.dynamic-shape",
}
model = CondBranchNonlocalVariables()


torch.export.export(model, example_args)
```

```
ExportedProgram:
    class GraphModule(torch.nn.Module):
        def forward(self, c_lifted_tensor_0: "f32[]", x:
"f32[6]"):
                add: "f32[6]" = torch.ops.aten.add.Tensor(x,
100)

                lift_fresh_copy: "f32[]" =
torch.ops.aten.lift_fresh_copy.default(c_lifted_tensor_0);
c_lifted_tensor_0 = None
            detach_: "f32[]" =
torch.ops.aten.detach_.default(lift_fresh_copy);
lift_fresh_copy = None

                add_1: "f32[6]" = torch.ops.aten.add.Tensor(x,
add);  x = add = None
            add_2: "f32[6]" = torch.ops.aten.add.Tensor(add_1,
detach_);  add_1 = detach_ = None
            return (add_2,)

Graph signature:
    # inputs
    c_lifted_tensor_0: CONSTANT_TENSOR target='lifted_tensor_0'
    x: USER_INPUT

    # outputs
    add_2: USER_OUTPUT

Range constraints: {}
```

```
# mypy: allow-untyped-defs
import torch

from torch.export import Dim
```

```
x = torch.randn(3, 2)
y = torch.randn(2)
dim0_x = Dim("dim0_x")

class CondOperands(torch.nn.Module):
    """
    The operands passed to cond() must be:
    - a list of tensors
    - match arguments of `true_fn` and `false_fn`

    NOTE: If the `pred` is test on a dim with batch size < 2, it
will be specialized.
    """

    def forward(self, x, y):
        def true_fn(x, y):
            return x + y

        def false_fn(x, y):
            return x - y

        return torch.cond(x.shape[0] > 2, true_fn, false_fn, [x,
y])

example_args = (x, y)
tags = {
    "torch.cond",
    "torch.dynamic-shape",
}
extra_inputs = (torch.randn(2, 2), torch.randn(2))
dynamic_shapes = {"x": {0: dim0_x}, "y": None}
model = CondOperands()


torch.export.export(model, example_args,
dynamic_shapes=dynamic_shapes)
```

```
ExportedProgram:
    class GraphModule(torch.nn.Module):
        def forward(self, x: "f32[s77, 2]", y: "f32[2]"):
             #
            sym_size_int_1: "Sym(s77)" =
torch.ops.aten.sym_size.int(x, 0)

                 gt: "Sym(s77 > 2)" = sym_size_int_1 > 2;
sym_size_int_1 = None

                 true_graph_0 = self.true_graph_0
            false_graph_0 = self.false_graph_0
            cond = torch.ops.higher_order.cond(gt, true_graph_0,
false_graph_0, (x, y));  gt = true_graph_0 = false_graph_0 = x =
y = None
            getitem: "f32[s77, 2]" = cond[0];  cond = None
            return (getitem,)

        class true_graph_0(torch.nn.Module):
            def forward(self, x: "f32[s77, 2]", y: "f32[2]"):
                        add: "f32[s77, 2]" =
torch.ops.aten.add.Tensor(x, y);  x = y = None
                return (add,)

        class false_graph_0(torch.nn.Module):
            def forward(self, x: "f32[s77, 2]", y: "f32[2]"):
                        sub: "f32[s77, 2]" =
torch.ops.aten.sub.Tensor(x, y);  x = y = None
                return (sub,)

Graph signature:
    # inputs
    x: USER_INPUT
    y: USER_INPUT

    # outputs
    getitem: USER_OUTPUT
```

```
Range constraints: {s77: VR[0, int_oo]}
```

```
# mypy: allow-untyped-defs
import torch

from functorch.experimental.control_flow import cond

class CondPredicate(torch.nn.Module):
    """
    The conditional statement (aka predicate) passed to cond()
must be one of the following:
      - torch.Tensor with a single element
      - boolean expression

    NOTE: If the `pred` is test on a dim with batch size < 2, it
will be specialized.
    """

    def forward(self, x):
        pred = x.dim() > 2 and x.shape[2] > 10

        return cond(pred, lambda x: x.cos(), lambda y: y.sin(),
[x])

example_args = (torch.randn(6, 4, 3),)
tags = {
    "torch.cond",
    "torch.dynamic-shape",
}
model = CondPredicate()


torch.export.export(model, example_args)
```

```
ExportedProgram:
    class GraphModule(torch.nn.Module):
        def forward(self, x: "f32[6, 4, 3]"):
                sin: "f32[6, 4, 3]" =
torch.ops.aten.sin.default(x);  x = None
            return (sin,)

Graph signature:
    # inputs
    x: USER_INPUT

    # outputs
    sin: USER_OUTPUT

Range constraints: {}
```

```
# mypy: allow-untyped-defs
import torch

class DynamicShapeConstructor(torch.nn.Module):
    """
    Tensor constructors should be captured with dynamic shape
inputs rather
    than being baked in with static shape.
    """

    def forward(self, x):
        return torch.zeros(x.shape[0] * 2)

example_args = (torch.randn(3, 2),)
tags = {"torch.dynamic-shape"}
model = DynamicShapeConstructor()


torch.export.export(model, example_args)
```

```
ExportedProgram:
    class GraphModule(torch.nn.Module):
        def forward(self, x: "f32[3, 2]"):
                zeros: "f32[6]" =
torch.ops.aten.zeros.default([6], device = device(type='cpu'),
pin_memory = False)
            return (zeros,)

Graph signature:
    # inputs
    x: USER_INPUT

    # outputs
    zeros: USER_OUTPUT

Range constraints: {}
```

Notes & Warnings

NOTE Note Tags: torch.cond, torch.dynamic-shape Support Level: SUPPORTED

NOTE Note Tags: torch.cond, torch.dynamic-shape Support Level: SUPPORTED

NOTE Note Tags: torch.cond, torch.dynamic-shape Support Level: SUPPORTED

NOTE Note Tags: torch.cond, torch.dynamic-shape Support Level: SUPPORTED

NOTE Note Tags: torch.cond, torch.dynamic-shape Support Level: SUPPORTED

NOTE Note Tags: torch.dynamic-shape Support Level: SUPPORTED

NOTE Note Tags: torch.dynamic-shape, python.control-flow Support Level: SUPPORTED

NOTE Note Tags: torch.dynamic-shape, torch.map Support Level: SUPPORTED

NOTE Note Tags: torch.dynamic-shape, python.builtin Support Level: NOT_SUPPORTED_YET

NOTE Note Tags: torch.dynamic-shape Support Level: SUPPORTED

NOTE Note Tags: torch.dynamic-shape Support Level: SUPPORTED

NOTE Note Tags: torch.dynamic-shape, python.data-structure, python.assert Support Level: SUPPORTED

NOTE Note Tags: torch.dynamic-shape Support Level: SUPPORTED

Detailed Documentation

cond_branch_class_method#

Original source code:

cond_branch_nested_function#

Original source code:

cond_branch_nonlocal_variables#

Original source code:

cond_operands#

Original source code:

cond_predicate#

Original source code:

dynamic_shape_constructor#

Original source code:

dynamic_shape_if_guard#

Original source code:

dynamic_shape_map#

Original source code:

dynamic_shape_round#

Original source code:

dynamic_shape_slicing#

Original source code:

dynamic_shape_view#

Original source code:

list_contains#

Original source code:

scalar_output#

Original source code: